

WILP, BITS PILANI DEEP NEURAL NETWORKS (AIML ZG511)

Session 10: Recurrent Neural Networks, BPTT, LSTM & GRU

Sequence Modeling, Backpropagation Through Time, Vanishing Gradients, and Gated Memory Cells

Module 7
Session 10 Reader
Full Worked Solutions

1 Sequence Modeling: Why Feedforward Networks Fail

Feedforward networks assume independent inputs. Sequence models handle ordered, variable-length inputs.

Standard feedforward networks require fixed vector inputs $x \in \mathbb{R}^d$ and process every sample independently. However, real-world signals like natural language, time-series temperatures, speech audio, and video frames depend heavily on sequential ordering and context.

The Assembly Line Memory

Imagine a conveyor belt worker inspecting parts. If every item is independent, a simple glance works. But if item 7 determines whether item 8 is defective, inspecting item 8 without knowing item 7 causes mistakes. Feedforward networks suffer from complete amnesia between steps.

Temperature Forecasting Numerical

Consider a simple autoregressive model predicting temperature T_{t+1} based on past steps:

$$T_{t+1} = 0.8T_t + 0.1T_{t-1} + 2.0$$

Given historical temperatures $T_1 = 20^\circ\text{C}$, $T_2 = 22^\circ\text{C}$, $T_3 = 25^\circ\text{C}$:

Step 1. Predict T_4 using past steps T_3 and T_2 :

$$T_4 = 0.8(25) + 0.1(22) + 2.0 = 20.0 + 2.2 + 2.0 = 24.2^\circ\text{C}$$

Step 2. Predict T_5 using $T_4 = 24.2$ and $T_3 = 25$:

$$T_5 = 0.8(24.2) + 0.1(25) + 2.0 = 19.36 + 2.5 + 2.0 = 23.86^\circ\text{C}$$

2 Vanilla Recurrent Neural Networks (RNN) & Forward Pass

RNNs pass a hidden state vector across time steps using shared weight matrices.

At each timestep t , the recurrent neural network accepts input vector $x_t \in \mathbb{R}^d$ and previous hidden state $h_{t-1} \in \mathbb{R}^m$, updating its memory via:

$$h_t = \tanh(W_{hx}x_t + W_{hh}h_{t-1} + b_h)$$

$$\hat{y}_t = \text{softmax}(W_{yh}h_t + b_y)$$

Unrolled Recurrent Neural Network Computational Graph

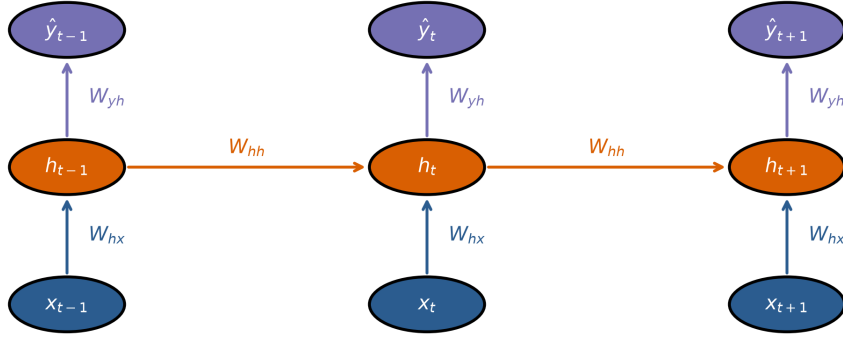


Figure 1: Unrolling a Recurrent Neural Network across time steps $t-1, t, t+1$. The weight matrices W_{hx}, W_{hh}, W_{yh} are shared across all timesteps.

♣ Parameter Sharing Across Time

Because sequences can be long, an RNN uses the **exact same** weight matrices W_{hx}, W_{hh}, W_{yh} at every single time step. This reduces parameter count drastically.

📎 Vanilla RNN Numerical Forward Pass

Let input dimension $d = 2$, hidden state dimension $m = 2$, and output dimension $K = 1$. Given parameters:

$$W_{hx} = \begin{bmatrix} 0.5 & 0.1 \\ -0.2 & 0.4 \end{bmatrix}, \quad W_{hh} = \begin{bmatrix} 0.8 & -0.1 \\ 0.3 & 0.5 \end{bmatrix}, \quad b_h = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}$$

Initial hidden state $h_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, and input sequence $x_1 = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}, x_2 = \begin{bmatrix} -0.5 \\ 1 \end{bmatrix}$.

Step 1. Compute pre-activation for timestep $t = 1$:

$$a_1 = W_{hx}x_1 + W_{hh}h_0 + b_h = \begin{bmatrix} 0.5(1) + 0.1(0.5) \\ -0.2(1) + 0.4(0.5) \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix} = \begin{bmatrix} 0.65 \\ -0.10 \end{bmatrix}$$

Step 2. Apply tanh activation for h_1 :

$$h_1 = \tanh(a_1) = \begin{bmatrix} \tanh(0.65) \\ \tanh(-0.10) \end{bmatrix} = \begin{bmatrix} 0.5716 \\ -0.0997 \end{bmatrix}$$

Step 3. Compute pre-activation for timestep $t = 2$:

$$\begin{aligned} a_2 &= W_{hx}x_2 + W_{hh}h_1 + b_h \\ &= \begin{bmatrix} -0.15 \\ 0.5 \end{bmatrix} + \begin{bmatrix} 0.4673 \\ 0.1216 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix} = \begin{bmatrix} 0.4173 \\ 0.5216 \end{bmatrix} \end{aligned}$$

Step 4. Apply tanh activation for h_2 :

$$h_2 = \tanh(a_2) = \begin{bmatrix} \tanh(0.4173) \\ \tanh(0.5216) \end{bmatrix} = \begin{bmatrix} 0.3945 \\ 0.4789 \end{bmatrix}$$

Keras SimpleRNN Parameter Counting

Consider Keras code: `model.add(SimpleRNN(units=3, input_shape=(4, 5)))`. Inputs have feature dimension $d = 5$, and hidden units $m = 3$.

Step 1. Compute W_{hx} parameters: $d \times m = 5 \times 3 = 15$.

Step 2. Compute W_{hh} parameters: $m \times m = 3 \times 3 = 9$.

Step 3. Compute bias vector b_h parameters: $m = 3$.

Step 4. Total RNN layer parameters = $15 + 9 + 3 = 27$.

3 Backpropagation Through Time (BPTT) & Vanishing Gradients

BPTT unrolls the network across time. Repeated matrix multiplication causes gradient decay.

To calculate $\frac{\partial L}{\partial W_{hh}}$, the gradient must flow backwards through time from step T down to step 1:

$$\frac{\partial L_T}{\partial h_k} = \frac{\partial L_T}{\partial h_T} \prod_{j=k+1}^T \frac{\partial h_j}{\partial h_{j-1}}$$

where $\frac{\partial h_j}{\partial h_{j-1}} = \text{diag}(1 - \tanh^2(a_j)) W_{hh}^T$.

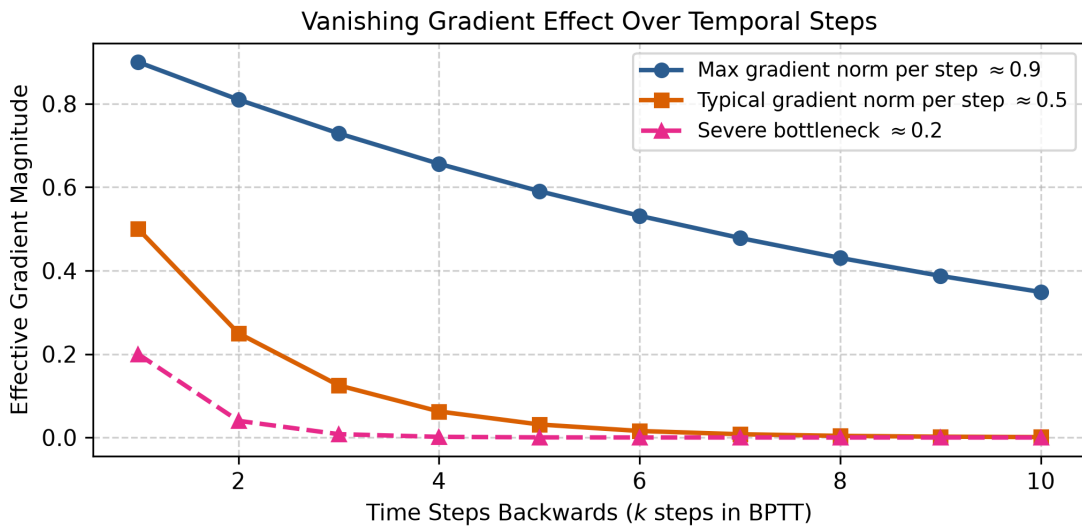


Figure 2: Effective gradient magnitude decaying exponentially as it backpropagates through time steps k .

! ? The Vanishing Gradient Trap

Because $\tanh'(x) \leq 1.0$ and weight matrices often have spectral radius < 1.0 , multiplying these terms 50 times results in factors like $0.5^{50} \approx 8.88 \times 10^{-16}$. The network completely forgets long-term context!

4 Long Short-Term Memory (LSTM) Networks

LSTMs maintain a Cell State highway controlled by three gating mechanisms.

An LSTM unit regulates information flow using three specialized gates:

$$\text{Forget Gate: } f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

$$\text{Input Gate: } i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

$$\text{Candidate State: } \tilde{C}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

$$\text{Updated Cell State: } C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

$$\text{Output Gate: } o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

$$\text{Hidden State: } h_t = o_t \odot \tanh(C_t)$$



Figure 3: LSTM Cell architecture showing the Cell State highway C_t and the three gating mechanisms (Forget, Input, Output).

LSTM Parameter Counting Formula

For an input dimension d and hidden dimension m , an LSTM cell contains 4 linear transformations (for $f, i, \tilde{C}, o$).

$$\text{Total Parameters} = 4 \times (m \times d + m \times m + m)$$

If input size $d = 100$ and hidden size $m = 200$:

$$\text{Total Parameters} = 4 \times (200 \times 100 + 200 \times 200 + 200) = 240,800$$

5 Gated Recurrent Units (GRU)

GRUs merge cell and hidden states into one vector using **Reset** and **Update** gates.

The GRU equations eliminate the cell state highway and operate directly on hidden state h_t :

$$\text{Reset Gate: } r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

$$\text{Update Gate: } z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

$$\text{Candidate Hidden State: } \tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h)$$

$$\text{Final Hidden State: } h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

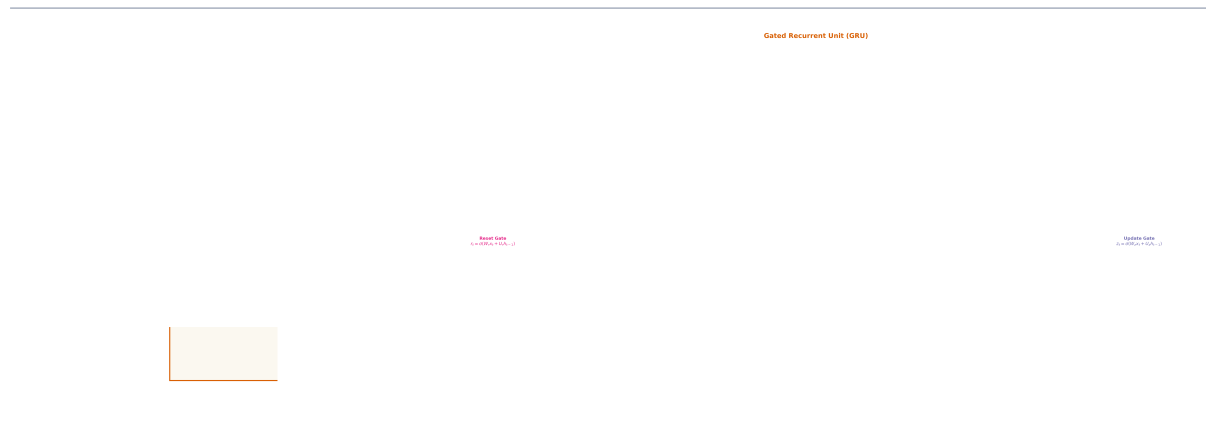


Figure 4: GRU Cell architecture showing Reset Gate r_t and Update Gate z_t .

Full GRU Numerical Calculation

Let input $x_t = 1.0$, past state $h_{t-1} = 0.5$. Parameters:

$$W_r = 0.2, U_r = 0.5, b_r = 0.0, \quad W_z = 0.8, U_z = -0.1, b_z = 0.1$$

$$W_h = 0.5, U_h = 0.9, b_h = -0.2$$

Step 1. Compute Reset Gate r_t :

$$a_r = 0.2(1.0) + 0.5(0.5) + 0.0 = 0.45 \implies r_t = \sigma(0.45) \approx 0.6107$$

Step 2. Compute Update Gate z_t :

$$a_z = 0.8(1.0) - 0.1(0.5) + 0.1 = 0.85 \implies z_t = \sigma(0.85) \approx 0.7006$$

Step 3. Compute Candidate Hidden State $\tilde{h}_t$:

$$\text{Gated state } r_t \odot h_{t-1} = 0.6107 \times 0.5 = 0.30535$$

$$a_h = 0.5(1.0) + 0.9(0.30535) - 0.2 = 0.5748 \implies \tilde{h}_t = \tanh(0.5748) \approx 0.5188$$

Step 4. Compute Final Hidden State h_t :

$$h_t = (1 - 0.7006)(0.5) + 0.7006(0.5188) = 0.1497 + 0.3635 = 0.5132$$

✓ LSTM vs GRU Comparison

Property	GRU	LSTM
Number of Gates	2 (Reset, Update)	3 (Forget, Input, Output)
State Vectors	1 (Hidden state h_t)	2 (Cell state C_t , Hidden state h_t)
Parameters	$3 \times (m \cdot d + m^2 + m)$	$4 \times (m \cdot d + m^2 + m)$
Computational Efficiency	Faster	Slower (more parameter heavy)

6 Self-Test & Pocket Recap

Test your understanding of sequence models, gating math, and parameter counting.

Self-Test Questions

1. **Question:** Why does an LSTM cell preserve gradients across long time steps better than a vanilla RNN?

Answer: The LSTM Cell State highway C_t uses additive updates ($C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$) rather than matrix multiplication. When $f_t \approx 1$, the gradient flows backward unchanged with factor 1.0, preventing exponential decay.

2. **Question:** Calculate the total parameter count for a GRU layer with input dimension $d = 10$ and hidden units $m = 20$.

Answer: GRU has 3 gate transformations. Parameters = $3 \times (m \cdot d + m^2 + m) = 3 \times (20 \cdot 10 + 20^2 + 20) = 3 \times (200 + 400 + 20) = 3 \times 620 = 1860$.